UAV Software Engineering: Concurrency and Thread Safety

Mutexes and Data Races

1 Theoretical Foundation: Data Races and Memory

In a multicore environment, a data race occurs when two or more threads access the same memory location simultaneously, and at least one of the accesses is a write (W operation). At the hardware level, reading a floating-point value while it is only partially written (memory tearing, this can happen because floats can take more than one clock to be written) results in garbage data (UB). In a drone's flight controller, feeding a corrupted altitude value into the PID loop will cause inconsitent (or even worse) motor commands.

2 UAV System Application: Synchronization Tools

To prevent data races, modern C++ provides three main synchronization primitives:

- `std::mutex`: Ensures mutual exclusion, meaning that only one thread can access the critical section (shared data such as a variable) at a time.
- `std::atomic`: Provides lock-free, hardware-level atomic operations, ideal for simple variables like flags or counters.
- `std::condition_variable`: Paired with a mutex, it blocks a thread until a specific system event occurs (e.g., waking up a thread only when a new telemetry packet arrives).

3 Implementation Details: Deadlocks and RAII

A manual call to `my_mutex.lock()` introduces a massive risk: if the thread forgets to call `unlock()` or if an exception is thrown inside the critical section, the mutex remains locked forever. This causes a deadlock, freezing the drone's flight controller.

To prevent this, the C++ Standard Library provides `std::lock_guard<std::mutex>`. By applying the RAII paradigm, `lock_guard` acquires the lock upon initialization and guarantees that the mutex is unlocked when the guard object goes out of scope, making the critical section completely exception-safe.

4 Advanced Architectures: Producer-Consumer and Data Queues

In high-frequency avionics systems, sensors (producers) generate data asynchronously and often at higher rates than flight controllers (consumers) can process them. Relying on a single shared variable can lead to data loss and deadlocks (we can see that by executing `../out/concurrency_condition.out` and noticing how the process never ends).

To solve this asynchronous timing mismatch, the **Producer-Consumer pattern** is implemented using a **thread-safe queue**. Instead of a single variable, data is pushed into a First-In-First-Out (FIFO) container like `std::queue`. Combined with `std::condition_variable` to notify the consumer only when new data is available, this architecture ensures that every telemetry packet is buffered and processed sequentially without wasting CPU cycles.

Please consider taking a look at the `../out/concurrency_condition_queue.cpp` and `../out/concurrency_condition_queue.out` for better understandings.